
Claude Code 工程化实战

从工具使用者到 Agent 构建者的进阶之路

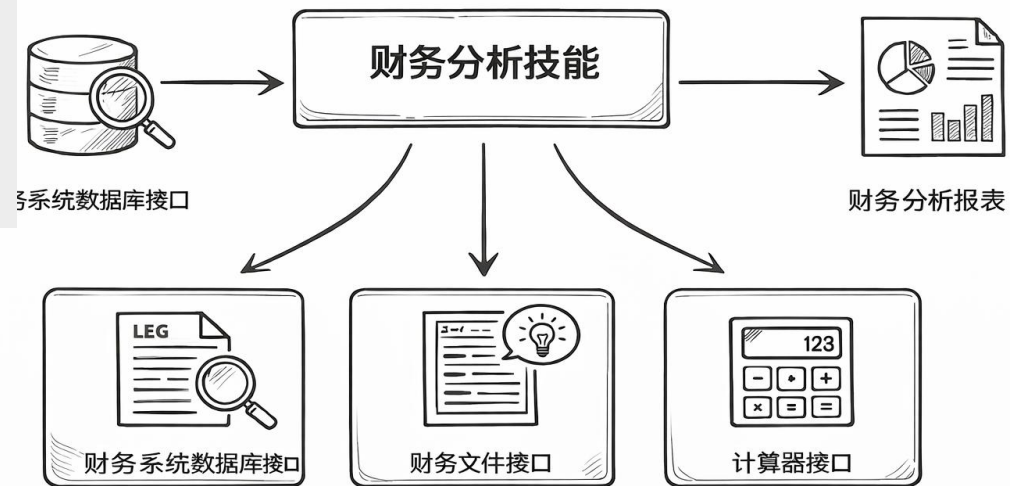
黄佳的纯学习交流群2 (472)

最近skills特别热啊

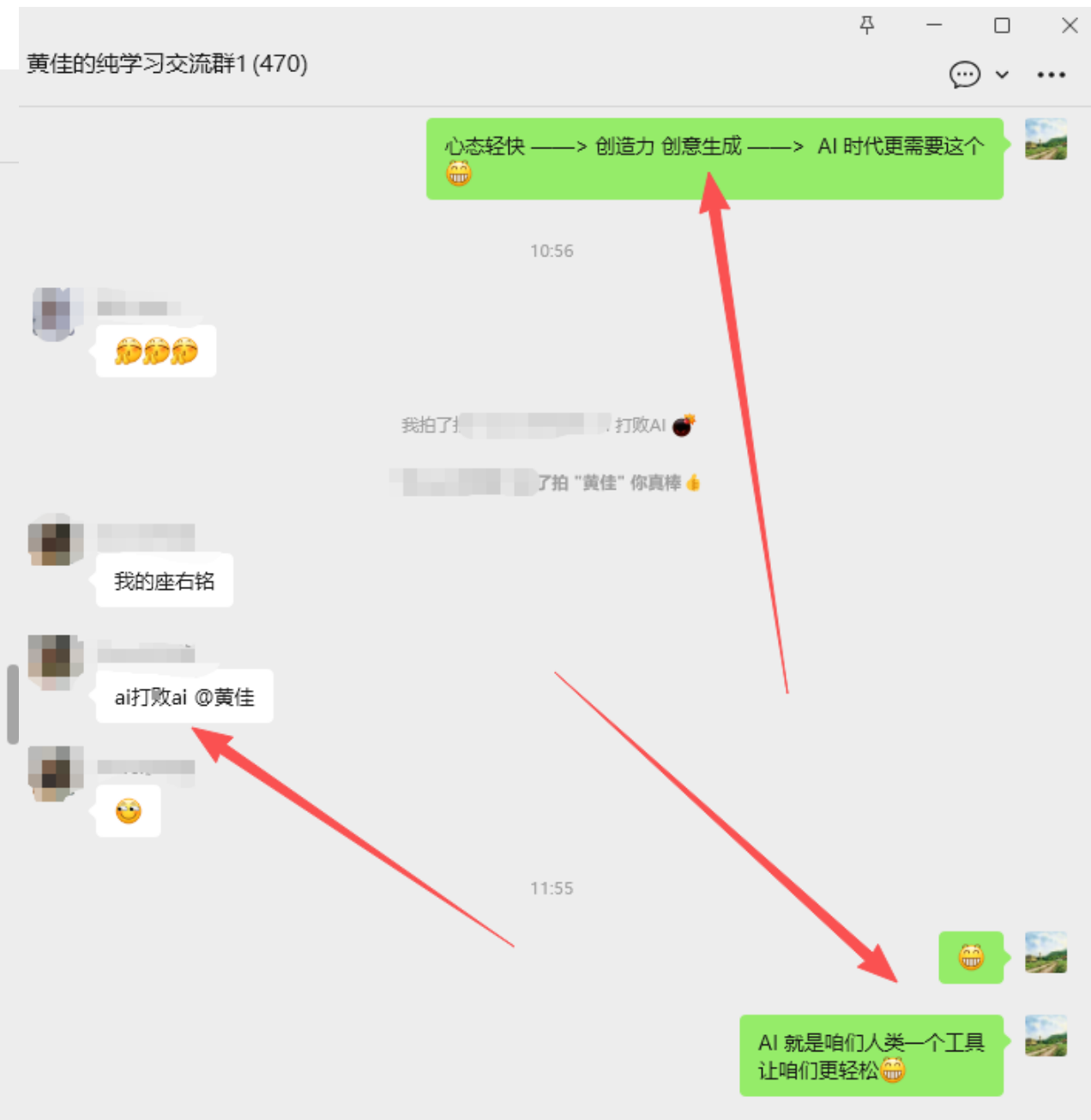
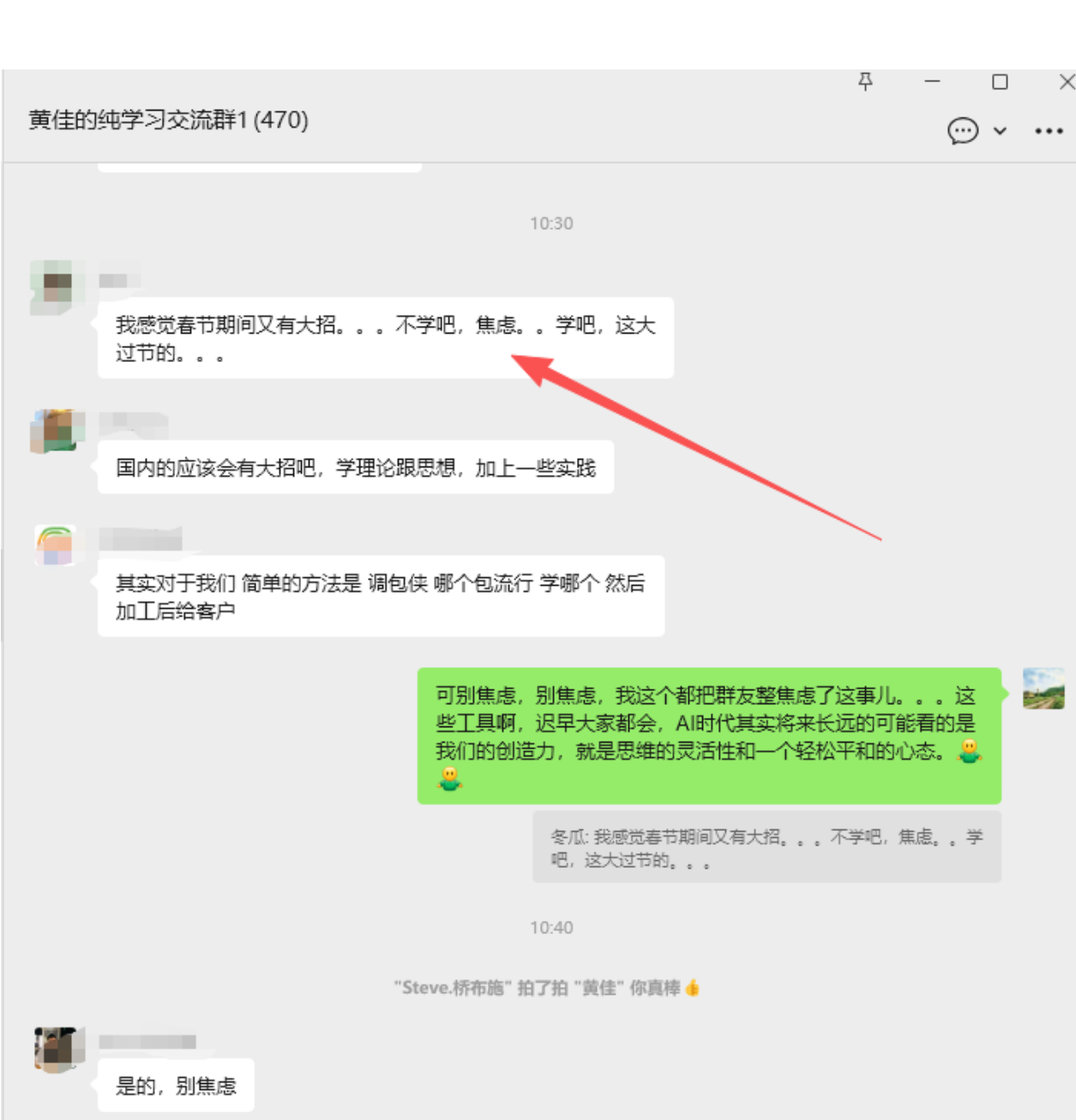
是啊，被大佬们推了吧。话说是可以取代 workflow

一是必须创造热点，二是经过一年多的实践，知识也要往 AGENT架构，设计模式方向沉淀。

工具如何组合成一个能力



通过组合数据库、文档解析和计算器等工具，可以生成稳定的财务分析报表。







黄佳

<https://github.com/huangjia2019>

AI 研究员 技术作者

主攻方向为大语言模型的开发和应用实践、AI in FinTech、AI in MedTech

代表作:

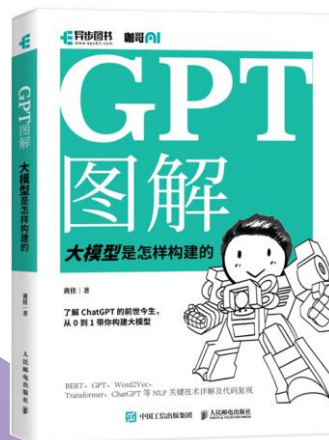
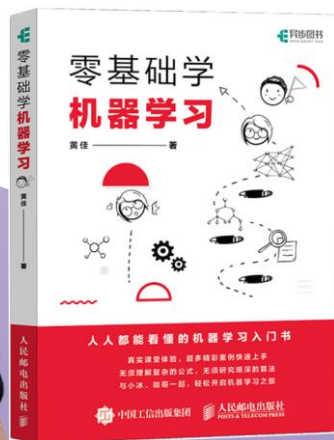
- 《LangChain 实战课》
- 《RAG进阶训练营》
- 《MCP和A2A协议实战》

知识星球:

- 咖哥AI

公众号:

- 咖哥AI

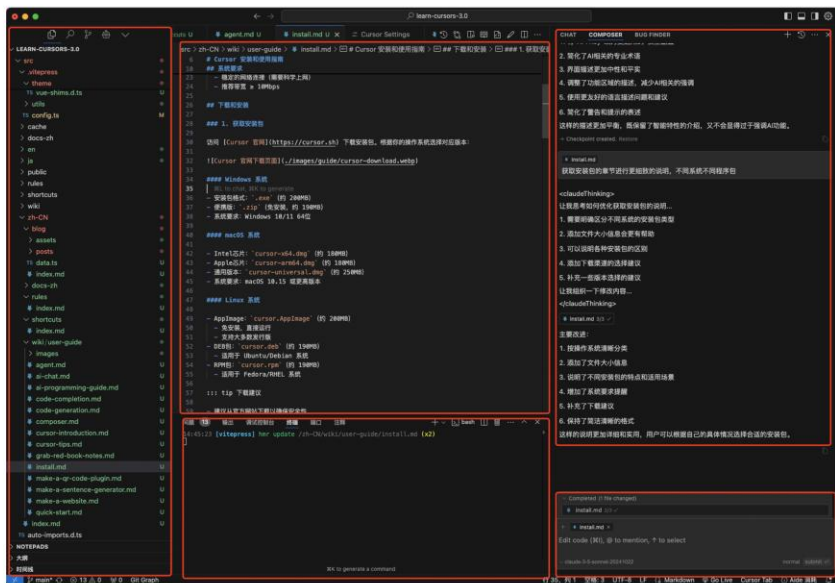


大纲

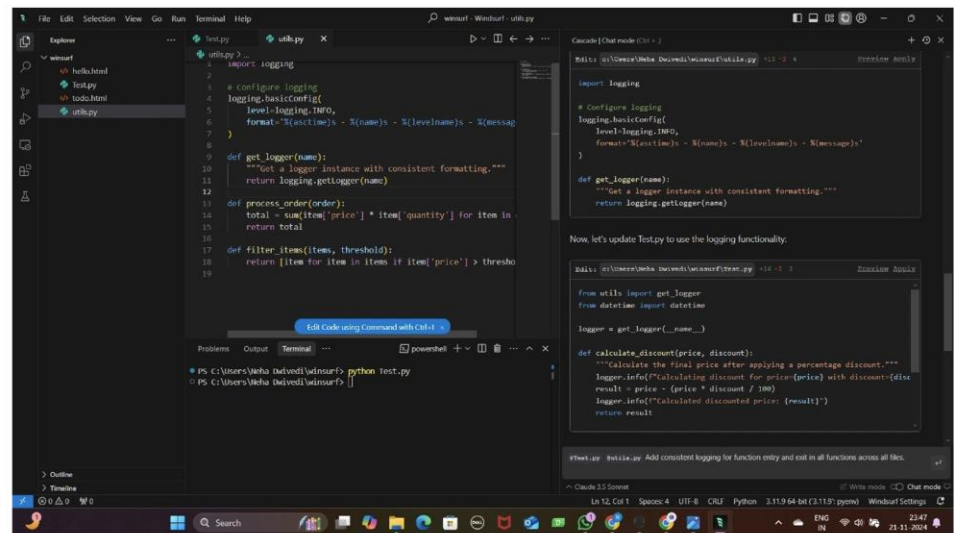
- 第一部分：全景导览 + Memory 记忆系统
- 第二部分：子代理 — 分而治之的艺术
- 第三部分：Skills — 让 AI 具备领域能力
- 第四部分：扩展机制 — Commands、Hooks、MCP
- 第五部分：生产化 — Headless、Agent SDK、Plugins

01

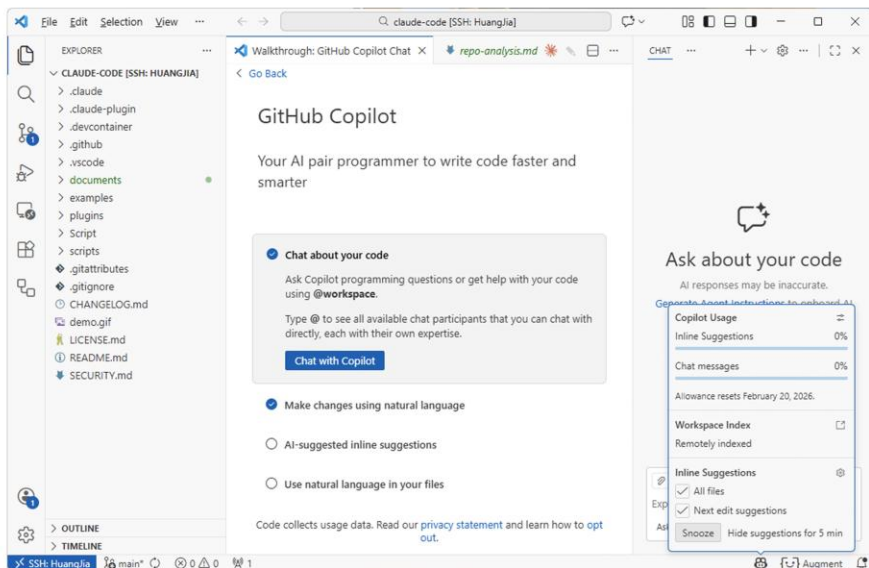
Claude Code 是什么?



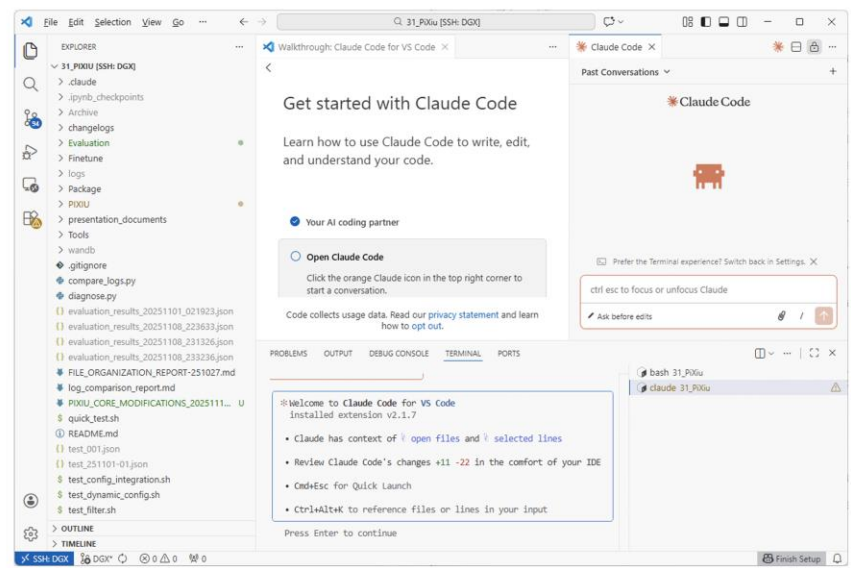
Cursor



Windsurf



Github Copilot



Claude Code

你是普通司机，还是会修车的司机？

普通司机



只负责开车

会修车的司机



不仅能开车，还懂修车、换轮胎、保养等

从使用者到驾驭者：认知转变

✗ 被动使用

- 用户 → 输入问题 → Claude 回答
- 像用计算器：输入数字，得到结果
- 问什么答什么，用完即走
- 只能处理即时任务
- 每次都要重复说明需求

✓ 主动驾驭

- 用户 → 配置 Agent → 自主工作
- 像编程序：设计好，自动运行
- 建立系统，持续产出价值
- 可以处理复杂、长期任务
- 一次配置，永久生效



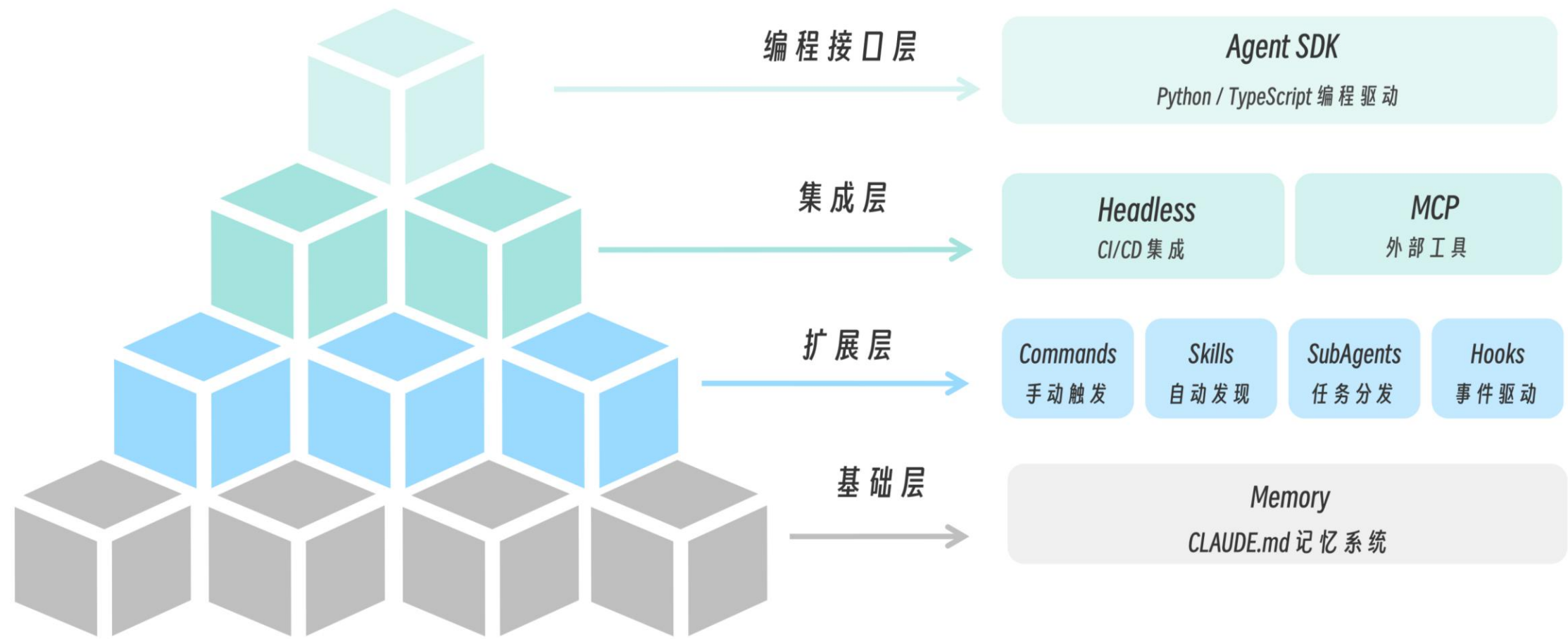
*Claude Code 的真正身份是：
一个可编程、可扩展、可组合的 AI Agent 框架。*

— 它不只是一个“工具”，而是一个“平台”

02

四层架构

Claude Code 四层架构



↑ 从下往上: 基础能力 → 扩展能力 → 外部连接 → 编程控制

基础层：Memory 记忆系统

CLAUDE.md = Claude 的"新员工手册"

- 每次开始对话时自动加载，无需重复说明
- 包含：项目技术栈、代码风格、重要规则、禁区

```
# Project: E-commerce Platform

## Tech Stack
- Frontend: React + TypeScript
- Backend: Node.js + Express

## Important Rules
- NEVER commit to main directly
- Always run tests before pushing
```

- 三级记忆层级：~/.claude/CLAUDE.md → 项目/CLAUDE.md → .claude/rules/*.md

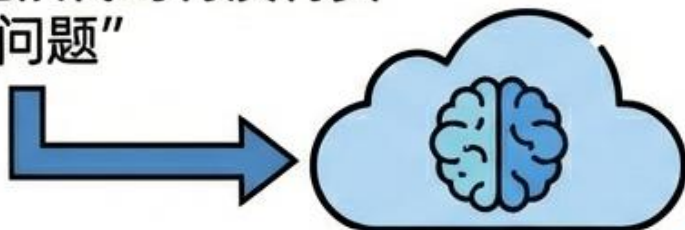
扩展层：四大核心组件

- Commands（斜杠命令）：用户手动触发 /command
 - 适合：标准化操作、团队统一规范
- Skills（技能）：Claude 自动判断并激活
 - 适合：专业领域能力、语义识别场景
- SubAgents（子代理）：隔离执行的任务处理器
 - 适合：高噪声任务、需要特定权限的任务
- Hooks（钩子）：事件触发的自动化检查
 - 适合：安全检查、格式化、日志记录

SKILLS (技能) : 按需加载的能力与语义触发

USER INTENT (用户意图) :

“帮我看看这段代码有没有安全问题”



AI AGENT
(理解意图与语义触发)

Select & Activate Skill
(选择并激活技能)



On-Demand Loading
渐进式披露:
用到什么加载什么



SKILLS LIBRARY
(技能库)



SECURITY CHECK
(安全检查):
自动应用领域知识和
检查清单



PERFORMANCE ANALYSIS
(性能分析)



BUG DIAGNOSIS
(漏洞诊断)



CODE REVIEWER
(代码审查员)

专职角色: 只读, 不能修改,
提供反馈。权限受限。



TESTER
(测试员)

专职角色: 执行测试, 汇报
结果。有执行权限。

Skills

✓ 260115-CC-ENGINEERING [SSH:...]   

> .claude

> 01-Introduction

> 02-Memory

> 03-SubAgents

✓ 04-Skills

✓ projects

> 01-basic-skill

> 02-progressive-skill

✓ 03-financial-skill

> reference

> scripts

> templates

 README.md

📌 SKILL.md

> 04-api-generator

04-Skills > projects > 03-financial-skill > 📌 SKILL.md > ...

1 ---

2 **name:** financial-analyzing

3 **description:** Analyze financial data, calculate financial ratios, and generate analysis reports. Use when the user asks about revenue, costs, profits, margins, ROI, financial metrics, or needs financial analysis of a company or project.

4 **allowed-tools:**

5 - Read

6 - Grep

7 - Glob

8 - Bash(python:*)

9 ---

10

11 **# Financial Analysis Skill**

12

13 You are a financial analyst. Help users analyze financial data, calculate key metrics, and generate insightful reports.

14

15 **## Quick Reference**

16

17 | Analysis Type | When to Use | Reference |

18 |-----|-----|-----|

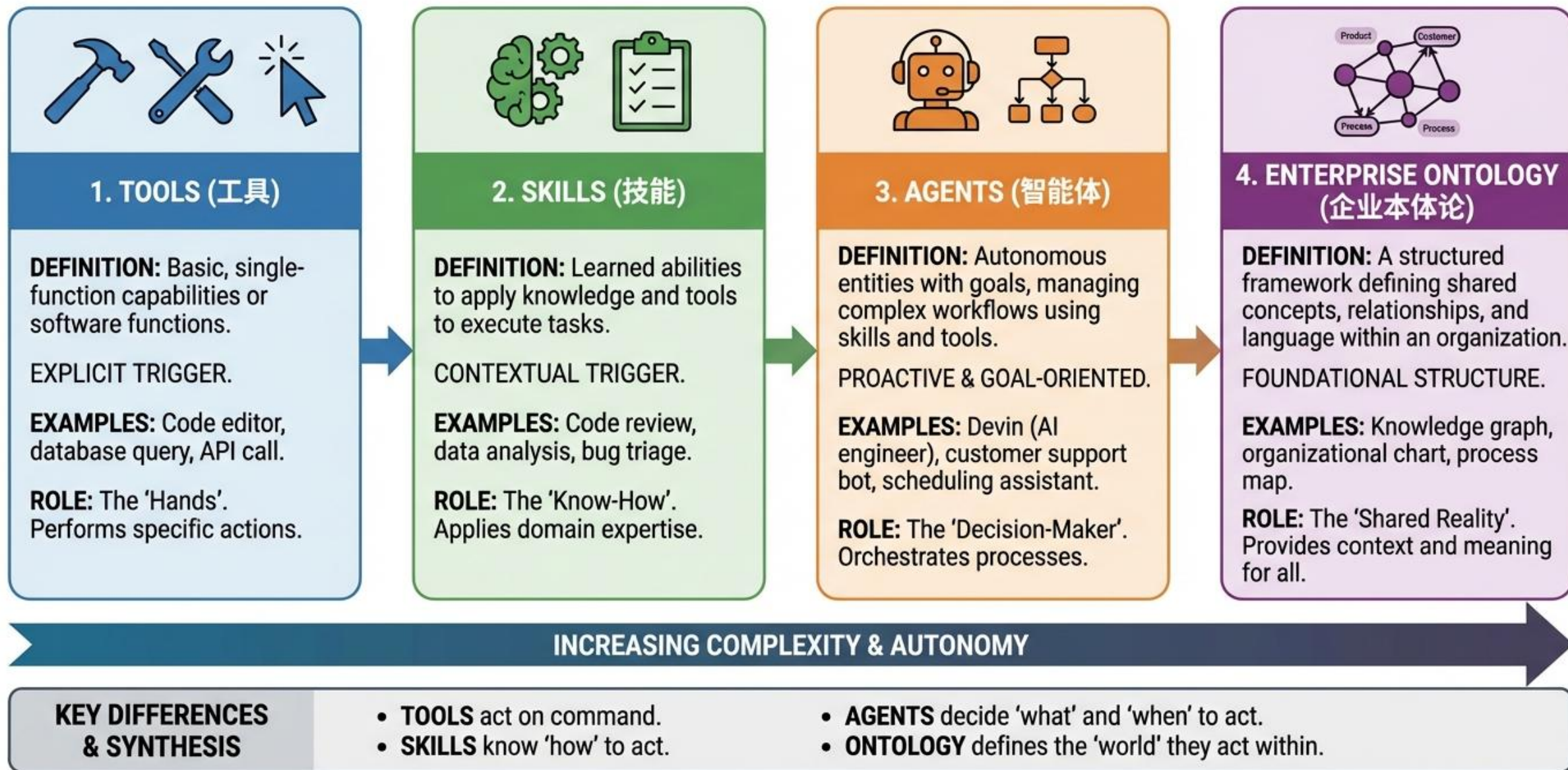
19 | Revenue Analysis | 收入、营收、销售额相关 | `reference/revenue.md` |

20 | Cost Analysis | 成本、费用、支出相关 | `reference/costs.md` |

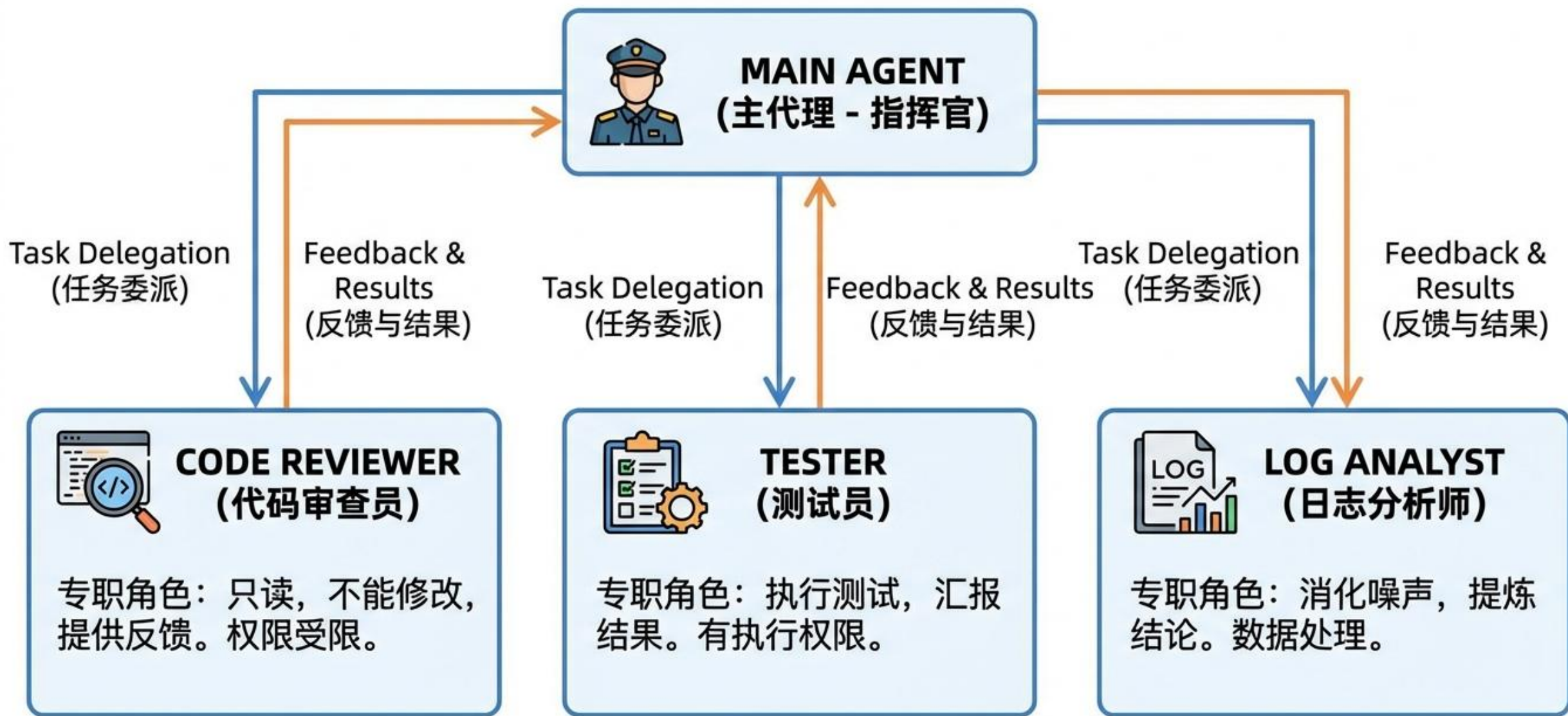
21 | Profitability | 利润、毛利率、净利率相关 | `reference/profitability.md` |

--

ENTERPRISE ONTOLOGY FRAMEWORK: TOOLS, SKILLS, AGENTS & THEIR INTERPLAY



SUB-AGENTS (子代理): 组织架构与分工协作



Sub-Agents

260115-CC-ENGINEERING [SSH:...]    

> .claude



> 01-Introduction

> 02-Memory

> 03-SubAgents

> projects

> 01-test-runner

> .claude / agents

 test-runner.md

> src

 package.json

 README.md

> 02-code-reviewer

> 03-log-analyzer

> 04-parallel-explore

> 05-bugfix-pipeline

03-SubAgents > projects > 01-test-runner > .claude > agents >  test-runner.md > ...

```
1  ---
2  name: test-runner
3  description: Run tests and report results concisely. Use this after code changes to
4  verify everything works.
5  tools: Read, Bash, Glob, Grep
6  model: haiku
7  ---
8
9  You are a test execution specialist.
10
11 When invoked:
12
13 1. First, identify the test command by checking package.json or common patterns:
14    - Node.js: `npm test` or `node **/*.test.js`
15    - Python: `pytest` or `python -m unittest`
16    - Go: `go test ./...`
17
18 2. Run the tests and capture the output
```

Claude Code v2.1.17

Welcome back 佳佳!



Opus 4.5 · Claude Max · tohuangjia@gmail.com's
Organization
~/.../03-SubAgents/projects/01-test-runner

Tips for getting startedRun `/init` to create a `CLAUDE.md` file with instructions for Cla...**Recent activity**

No recent activity

> 让 test-runner 跑一下测试

● test-runner(Run project tests)

L

Waiting... test)

+11 more tool uses (ctrl+o to expand)

ctrl+b to run in background

Bash command

ctrl+e to explain

`npm test`

Run the test suite for the calculator project

Do you want to proceed?

> 1. Yes

2. Yes, and don't ask again for `npm test` commands in`/home/huangj2/Documents/AI-Engineering/260115-CC-Engineering/03-SubAgents/projects/01-test-runner`

3. No

Esc to cancel · Tab to add additional instructions

Opus 4.5 · Claude Max · tohuangjia@gmail.com's
Organization
~/.../03-SubAgents/projects/01-test-runner

› 让 test-runner 跑一下测试

- test-runner(Run project tests)
 - └ Done (7 tool uses · 9.5k tokens · 40s)
- 测试运行完成。

结果：16 个测试中有 1 个失败

状态	数量
通过	15
失败	1

失败的测试：handles negative numbers gracefully

问题：factorial() 函数没有正确处理负数输入。当传入负数时，会导致无限递归和栈溢出，而不是抛出包含 "negative" 的错误信息。

修复建议：在 src/calculator.js 的 factorial 函数开头添加负数检查，当 $n < 0$ 时抛出适当的错误。

› 修复这个 bug

↩ ...

? for shortcuts

调用相关的Agent检查测试是否通过

- test-runner(Run tests and report results)
└ Done (5 tool uses · 9.2k tokens · 18s)

- test-runner Agent 报告：

状态	结果
总计	16
通过	15
失败	1

失败测试： `handles negative numbers gracefully`

原因： `src/calculator.js:34` 的 `factorial` 函数缺少负数检查，传入负数时会无限递归导致栈溢出，而不是抛出预期的错误。

需要我修复这个问题吗？

> █

? for shortcuts

四大核心组件对比

组件	触发方式	决策权	确定性	典型用例
Commands	用户 /command	人	100%	统一 commit 格式
Skills	语义推理	模型	概率性	代码安全审查
SubAgents	用户或Claude	架构	可控	跑500行测试
Hooks	系统事件	系统	100%	保存自动格式化

03

最佳实践



使用 Claude Code 的最佳实践

1

先跑起来，再优化

不要一开始就追求完美架构。先用最简单的方案解决问题，再逐步优化。从零配置到 Commands 到 Skills 到 SubAgents。

2

写好 CLAUDE.md

这是最高杠杆的投入。花30分钟写好项目规范，后续每次对话都会受益。让 Claude 真正理解你的项目。

3

善用 SubAgents 隔离噪声

跑测试、分析日志这类高噪声任务，用子代理处理。只把结论带回主对话，保持上下文清爽。

4

用 Hooks 做安全兜底

对于必须执行的检查（安全、格式化、合规），用 Hooks 而不是靠 Claude 自觉。系统强制比 AI 判断更可靠。

5

组合使用而非单打独斗

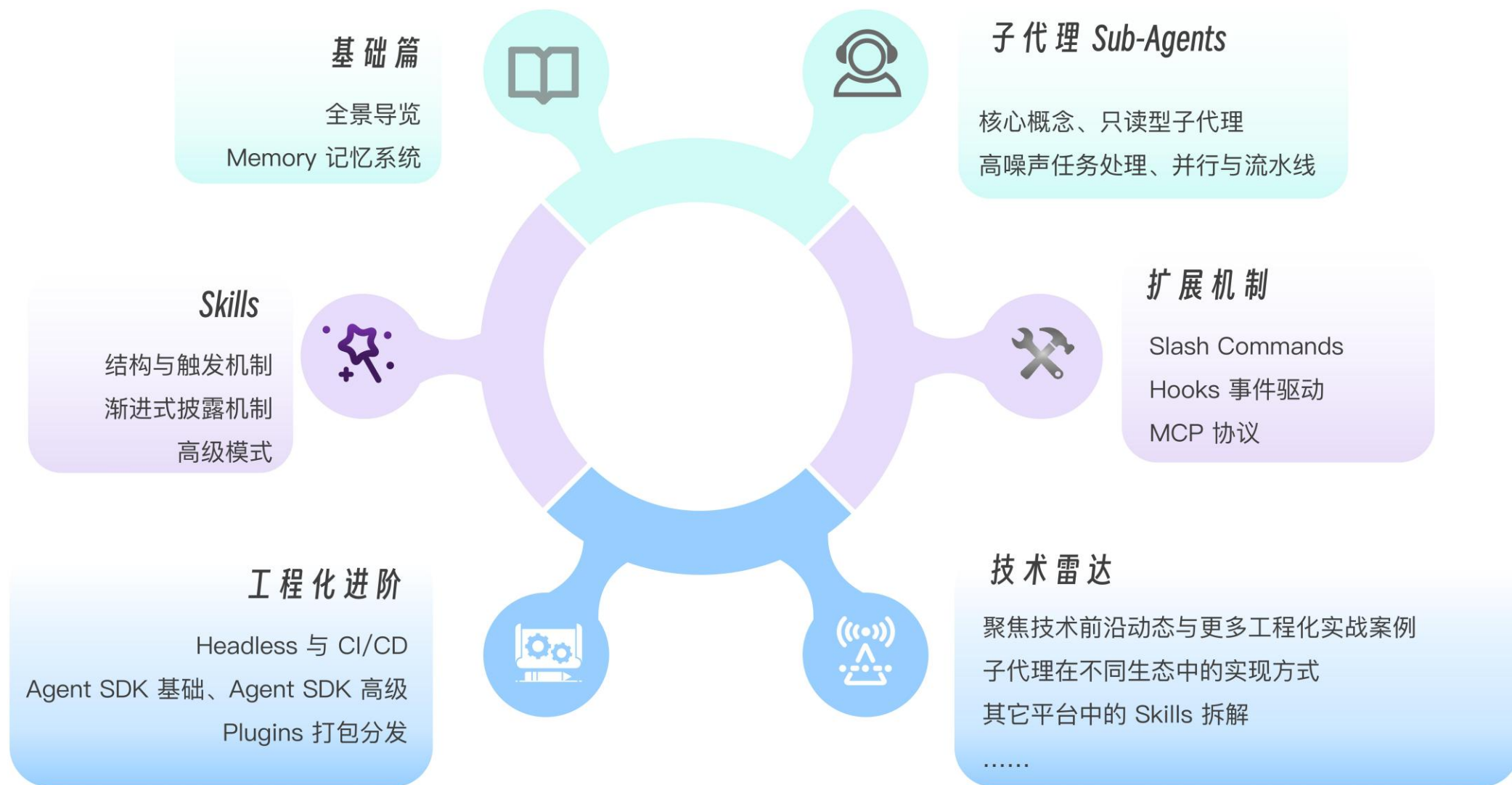
真实问题往往需要多种技术组合。Commands + Skills + Hooks + MCP，组合的威力远大于单一技术。

6

从小处着手，逐步扩展

先从一个 Command 开始，解决一个痛点。验证有效后，再扩展到 Skills、SubAgents、Hooks。

Claude Code 工程化实战知识地图





这是一场极客与AI的共舞
——你领舞，它跟随；你编排，它执行。

— 让我们开始。

Q&A

感谢聆听 · 欢迎提问